Optimization Algorithms Final Project

A Comprehensive Analysis of Evolutionary and Swarm Intelligence Methods for Neural Network Training

Group 42
Afonso Fonseca 20241781
David Morais 20241724
Martinho Abreu 20241687
Pedro Jorge 20241819

June 2026

Abstract

Modern machine learning relies on gradient-based optimization to train artificial neural networks. While efficient, such methods are prone to entrapment in local optima within highly non-convex loss landscapes. This report presents a derivative-free alternative: we formulate neural network weight optimization as a global, continuous search problem and solve it using two metaheuristic algorithms—a Genetic Algorithm (GA) and Particle Swarm Optimization (PSO). Both were applied to train a Multilayer Perceptron (MLP) on the Oxford Parkinson's Disease Detection dataset, a binary classification task with significant class imbalance. We provide a rigorous comparative analysis of both algorithms across 10 independent runs, including a dedicated operator comparison for the GA (two crossover operators, two mutation operators, one additional selection operator, and two initialization methods, each evaluated over 5 runs with standard deviation bands) and a full statistical evaluation using paired t-tests and Wilcoxon signed-rank tests.

Contents

1. Introduction

1.1 Problem Statement

Training an artificial neural network is, at its core, a high-dimensional continuous optimization problem: find the weight vector $\boldsymbol{\theta} \in \mathbb{R}^n$ that minimizes prediction error on a training set. Backpropagation-based gradient descent is the dominant solution, but it suffers from two structural weaknesses: (i) it requires differentiability of the loss function, and (ii) it is sensitive to initialization and can converge to local optima in non-convex landscapes. Population-based metaheuristics avoid both limitations by maintaining a diverse pool of candidate solutions and searching stochastically.

1.2 Project Scope and Objectives

This project has four core objectives, directly aligned with the course guidelines:

1. **Network representation:** bridge scikit-learn's `MLPClassifier` with custom optimization libraries, enabling seamless extraction and injection of flat weight vectors.
2. **Algorithm implementation:** engineer a full GA (with 2 crossover operators, 2 mutation operators, 1 additional selection operator beyond class, and 2 initialization methods) and a full PSO from scratch.
3. **Operator comparison:** empirically contrast the operators against each other, with results averaged over multiple runs to ensure statistical reliability.
4. **Comparative validation:** rigorously contrast GA and PSO convergence and final performance across 10 independent runs using paired statistical tests.

2. Dataset

We use the **Oxford Parkinson's Disease Detection dataset**, a widely studied benchmark in medical signal processing. The dataset comprises 195 biomedical voice recordings from 31 individuals (23 with Parkinson's disease, 8 healthy controls). Each sample is characterised by 22 continuous acoustic features, including:

- **Frequency measures:** MDVP:Fo(Hz), MDVP:Fhi(Hz), MDVP:Flo(Hz)
- **Jitter measures:** MDVP:Jitter(%), MDVP:Jitter(Abs), MDVP:RAP, MDVP:PPQ, Jitter:DDP
- **Shimmer measures:** MDVP:Shimmer, MDVP:Shimmer(dB), APQ3, APQ5, MDVP:APQ, Shimmer:DDA
- **Noise / nonlinear features:** NHR, HNR, RPDE, DFA, D2, spread1, spread2, PPE

The binary label `status` takes value 1 (Parkinson's) or 0 (healthy). The class distribution is strongly imbalanced: **147 samples (75.4%)** are positive and only **48 (24.6%)** are healthy. The dataset was provided pre-standardised.

Train/Test Split

Data is partitioned 80/20 using stratified sampling (`stratify=y`) to preserve the class ratio in both splits. Optimization runs exclusively on the training partition; all reported metrics are computed on the held-out test set.

3. Network Representation and Fitness Evaluation

3.1 Mathematical Formulation

A Multilayer Perceptron computes the transformation at layer l as:

$$\mathbf{a}^{(l)} = \sigma\big(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\big)$$

where $\mathbf{W}^{(l)}$ is the weight matrix, $\mathbf{b}^{(l)}$ the bias vector, and σ the activation function (ReLU for hidden layers, softmax for the output). Our architecture is fixed:

Layer	**Neurons**	**Parameters**
Input	22	—
Hidden 1	10	$22 \times 10 + 10 = 230$
Hidden 2	5	$10 \times 5 + 5 = 55$
Output	1	$5 \times 1 + 1 = 6$
Total		**291**

A **solution** is the flat real-valued vector $\boldsymbol{\theta} = (w_1, \ldots, w_{291}) \in \mathbb{R}^{291}$.

3.2 Implementation (`network.py`)

The function `create_network()` instantiates a pre-trained `MLPClassifier` skeleton via `partial_fit` on dummy data, initialising all internal weight shapes without performing meaningful learning. `get_weights(model)` flattens all `coefs_` and `intercepts_` into $\boldsymbol{\theta}$. `set_weights(model, θ)` reverses this, reshaping $\boldsymbol{\theta}$ back into the network's parameter matrices layer by layer.

The function `generate_solution(model, method, rng)` produces a random initial weight vector using one of two strategies detailed in Section 4.1.

3.3 Fitness Function and Metric Justification

Standard accuracy is inappropriate for this dataset: a trivial classifier that always predicts Parkinson's achieves **75.4% accuracy** with zero discriminative power. We therefore use the **Macro F1-Score**:

$$F_1^{\text{macro}} = \frac{1}{2}\Big(F_1^{(0)} + F_1^{(1)}\Big), \qquad F_1^{(k)} = \frac{2\,\text{precision}^{(k)} \cdot \text{recall}^{(k)}}{\text{precision}^{(k)} + \text{recall}^{(k)}}$$

This metric weights both classes equally, directly penalising poor recall on the minority (healthy) class. In a clinical Parkinson's screening context, missing a healthy patient (false positive) and missing a Parkinson's patient (false negative) are both costly, making class-balanced evaluation essential. Both algorithms *maximise* fitness: $\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} F_1^{\text{macro}}(\boldsymbol{\theta})$.

4. Genetic Algorithm Architecture

The GA, implemented in `ga.py`, simulates Darwinian evolution by iteratively applying selection, crossover, mutation, and elitism to a population of weight vectors.

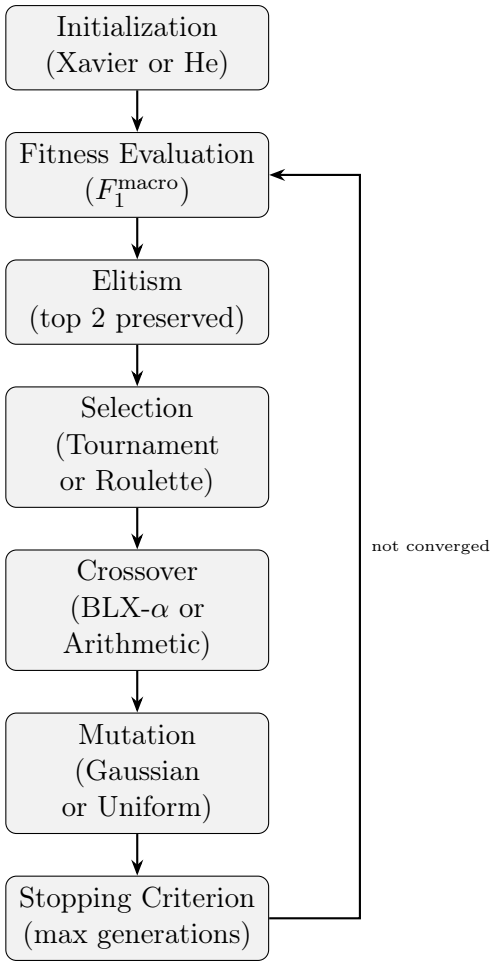

Figure 1: Genetic Algorithm lifecycle. At each generation, the top 2 individuals are preserved by elitism before the rest of the population is replaced through selection, crossover, and mutation.

4.1 Initialization Methods

Both methods are implemented inside `generate_solution(model, method, rng)` and produce the initial population for the GA (and the initial swarm positions for PSO). The choice of initialization determines the starting distribution of the search and can significantly affect convergence speed.

- **Xavier Uniform** (`method="uniform"`, *new*): weights sampled per layer from

$$\mathcal{U}\left[-\sqrt{6/(n_{\text{in}}+n_{\text{out}})},\ \sqrt{6/(n_{\text{in}}+n_{\text{out}})}\right]$$

  Designed to keep the variance of activations and gradients approximately equal across layers; widely used with sigmoid and tanh activations.

- **He Normal** (`method="normal"`, *new*): weights sampled per layer from $\mathcal{N}(0,\ 2/n_{\text{in}})$. Derived specifically for ReLU activations; the larger scale factor compensates for the fact that ReLU zeroes out half of its inputs on average, preventing vanishing activations in deep networks.

Both strategies are **layer-aware**: the scale is computed from the dimensions of each individual weight matrix rather than using a global constant. This is in contrast to

the naive random initialization used in the Knapsack problem in class (simple uniform $\mathcal{U}(0,1)$ or $\{0,1\}$ sampling), which has no theoretical grounding for real-valued continuous problems.

Initialization Comparison

The two methods are directly compared through the operator experiment (Section 7): Config A uses Xavier Uniform and Config B uses He Normal. Across 5 runs, Xavier Uniform (Config A) achieved a mean test F1-macro of 0.7751 ± 0.0736, compared to 0.7645 ± 0.0867 for He Normal (Config B). Although the difference is small, Xavier Uniform produced lower variance, suggesting it generates more consistently useful starting populations on this dataset. This is plausible: the network uses ReLU activations, but the dataset is small (156 training samples) and the network shallow, so the distinction between Xavier and He is less critical than in deep architectures. The tighter scale of Xavier Uniform ($\approx \pm 0.37$ for the first layer) may prevent early-generation individuals from producing saturated activations, giving the optimizer a better signal from generation 0.

4.2 Selection Operators

- **Tournament Selection** (*implemented in class*): K=3 individuals are sampled uniformly without replacement; the fittest among them is selected as a parent. Selection pressure is tunable via K without any fitness rescaling, and it is naturally robust to fitness noise—a useful property given that F1-macro is a discrete-valued function of predictions.
- **Rank-based Roulette Wheel** (*new extra operator, beyond class*): individuals are ranked by fitness and selection probability is proportional to ordinal rank. This mitigates premature convergence caused by dominant super-individuals, since fitness differences are replaced by rank differences. Unlike Tournament selection, it provides softer selection pressure and is better suited to later generations when population diversity is already reduced.

4.3 Crossover Operators

Both operators are adapted for real-valued vectors. The binary single-point crossover used in class for the Knapsack problem is not meaningful here, as swapping arbitrary subsets of continuous weights does not preserve any useful structure.

- **Arithmetic Crossover**: offspring are convex combinations of the parents,

$$c^{(1)} = \beta p^{(1)} + (1-\beta)p^{(2)}, \quad c^{(2)} = (1-\beta)p^{(1)} + \beta p^{(2)}, \quad \beta \sim \mathcal{U}(0,1).$$

  Offspring always lie strictly within the line segment between parents—exploitative and conservative, analogous in spirit to the interpolative nature of uniform crossover discussed in class.

- **BLX-α (Blend Crossover)**: for each gene i, offspring are drawn from an *extended* interval,

$$C_i \sim \mathcal{U}[\min(p_1,p_2) - \alpha d,\ \max(p_1,p_2) + \alpha d], \quad d = |p_1 - p_2|, \quad \alpha = 0.5.$$

The α factor allows search *outside* the parental hyper-rectangle, providing exploration beyond what the current population can represent through interpolation alone. This is the key advantage over arithmetic crossover in high-dimensional spaces where the population may converge too early.

Comparison to the Class Operator

In class, crossover was implemented as **one-point crossover** on binary strings: a random cut-point splits both parent chromosomes and the tails are swapped. This is well-suited to the Knapsack problem, where genes are binary inclusion flags and the positional meaning of a cut is interpretable. Applied naively to a real-valued weight vector it would produce offspring that inherit contiguous *blocks* of weights from each parent—a structurally arbitrary partition with no geometric meaning in the weight space. Arithmetic and BLX-α crossover replace this with operations that are geometrically meaningful in $\mathbb{R}^{291}$: offspring lie on the line segment between parents (arithmetic) or in an extended neighbourhood of it (BLX-α), preserving the continuous topology of the search space.

4.4 Mutation Operators

Both mutation operators perturb individual genes stochastically. Unlike the bit-flip mutation used in class for binary representations, these operators must produce meaningful perturbations in continuous space.

- **Gaussian Mutation**: adds noise $\delta \sim \mathcal{N}(0, \sigma^2)$ to each gene independently with probability p_m per gene. The Gaussian distribution is unbounded but concentrated near zero, producing smooth, fine-grained local perturbations. This is appropriate when the fitness landscape is approximately smooth and local refinement is desired.
- **Uniform Mutation**: adds bounded noise $\delta \sim \mathcal{U}(-\sigma, \sigma)$ to each gene with probability p_m. In class, mutation flipped bits; here the same role is played by a bounded additive perturbation. The hard bounds on δ prevent catastrophic large jumps, and the flat distribution provides equal probability of any perturbation magnitude within range, which can be useful for escaping flat fitness regions.

Comparison to the Class Operator

In class, mutation was implemented as **bit-flip mutation**: each binary gene is independently flipped with probability p_m, producing a maximally disruptive change ($0 \to 1$ or $1 \to 0$) with no notion of magnitude. This is appropriate for binary search spaces where there is no meaningful interpolation between gene values. In a continuous weight space, an equivalent hard replacement would destroy any fine structure the optimizer has built up in the weight vector. Both Gaussian and Uniform mutation instead apply *additive perturbations*, preserving the current value as a baseline and modifying it by a controlled amount. This makes the operators sensitive to the current state of the search—small σ produces local refinement, large σ produces exploration—a property that has no analogue in bit-flip mutation.

4.5 Elitism

The top 2 individuals by fitness are copied unchanged into the next generation, guaranteeing monotone improvement of the population best across generations.

5. Particle Swarm Optimization Architecture

PSO, implemented in `pso.py`, models a swarm of particles that collectively search the weight space by balancing personal experience and social influence.

5.1 Why PSO?

PSO was selected over alternatives (ABC, GWO, WOA, DE, etc.) for three reasons: (i) it is the most natural fit for continuous real-valued problems, sharing the same search space structure as our weight vector; (ii) it is well-studied with a clear mathematical framework; and (iii) its two key hyperparameters (c_1, c_2) have well-understood intuitive interpretations (cognitive vs. social pull), making tuning tractable without exhaustive grid search.

5.2 Kinematic Update Equations

At each iteration t, particle i updates its velocity and position:

$$\mathbf{v}_i(t+1) = w(t)\,\mathbf{v}_i(t) + c_1\, r_1 \odot \big(\mathbf{p}_i - \mathbf{x}_i(t)\big) + c_2\, r_2 \odot \big(\mathbf{g} - \mathbf{x}_i(t)\big) \tag{1}$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1) \tag{2}$$

where $\mathbf{p}_i$ is particle i's personal best, $\mathbf{g}$ is the global best, $r_1, r_2 \sim \mathcal{U}(0,1)^{291}$ are element-wise random vectors, and $\odot$ denotes element-wise multiplication.

5.3 Inertia Decay and Numerical Stability

The inertia weight decays linearly to balance exploration and exploitation:

$$w(t) = w_{\text{start}} - (w_{\text{start}} - w_{\text{end}}) \cdot \frac{t}{T}$$

High w in early iterations promotes broad exploration; as w decreases particles converge toward known good regions. Preliminary runs with constant w=0.9 exhibited persistent oscillation without convergence, confirming the necessity of decay.

Velocity clamping prevents swarm explosion in high-dimensional spaces:

$v_{i,j} \leftarrow \text{clip}(v_{i,j},\, -v_{\max},\, v_{\max})$.

Position clamping to [pos_min, pos_max] = $[-2, 2]$ maintains numerical stability of the network weights, a range chosen to be consistent with the magnitudes produced by Xavier initialization.

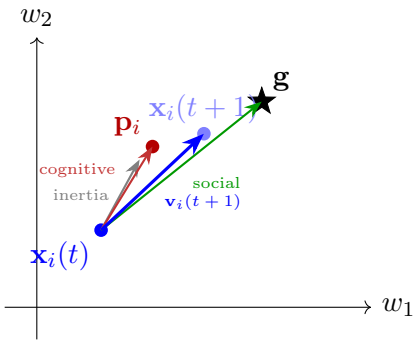

Figure 2: 2D projection of PSO particle movement (Eqs. 1–2). The new velocity (blue) is a weighted sum of three components: inertia (gray), cognitive pull toward the personal best $\mathbf{p}_i$ (red), and social pull toward the global best $\mathbf{g}$ (green). In 291 dimensions the same decomposition applies element-wise.

6. Experimental Setup and Hyperparameter Optimization

All experiments were structured to give both algorithms an equal computational budget (generations × population/swarm size). Hyperparameter ranges were explored on a held-out split and then fixed for the main comparative experiment.

6.1 Hyperparameter Grid Explored

Table 1: Hyperparameter search ranges explored for both algorithms. Final selected values are shown in bold.

Algorithm	Parameter	Range Explored	Final Value
GA	Population size	30, 50, **60**, 100	**60**
	Generations	50, **80**, 150	**80**
	Crossover probability	0.6, 0.7, **0.8**	**0.80**
	Mutation probability	0.1, 0.2, **0.25**	**0.25**
	Gene mutation rate	0.05, 0.10, **0.15**	**0.15**
	Mutation scale σ	0.05, **0.10**, 0.20	**0.10**
	BLX-α	0.3, **0.5**, 0.7	**0.50**
PSO	Swarm size	30, 50, **60**, 100	**60**
	Iterations	50, **80**, 150	**80**
	c_1 (cognitive)	1.2, **1.6**, 2.0	**1.60**
	c_2 (social)	1.2, **1.6**, 2.0	**1.60**
	w_{start}	**0.9**, 0.8	**0.90**
	w_{end}	**0.4**, 0.2	**0.40**

7. GA Operator Comparison

7.1 Experimental Design

To isolate the effect of operator choice from between-run variability, each configuration was evaluated over **5 independent runs** with different random seeds. Results are reported

as mean ± std. Two complete operator pipelines were compared over 50 generations. Config A groups the new operators implemented for this project; Config B groups the class-derived operators adapted to continuous spaces:

Table 2: The two GA operator configurations compared. Config A uses the newly implemented operators; Config B uses the operators developed/adapted from class.

Config	**Init**	**Selection**	**Crossover**	**Mutation**
A	Xavier Uniform	Tournament ($k = 3$)	BLX-α ($\alpha = 0.5$)	Gaussian ($\sigma = 0.1$)
B	He Normal	Roulette Wheel	Arithmetic	Uniform ($\sigma = 0.1$)

7.2 Results and Discussion

Table 3: Operator comparison results averaged over 5 runs. Config A outperforms Config B on both training and test fitness, indicating that BLX-α with Gaussian mutation achieves better generalisation when results are averaged across seeds.

Configuration	**Train F1-macro**	**Test F1-macro**
Config A (BLX-α / Gaussian / Tournament / Xavier)	0.8889 ± 0.0177	$\mathbf{0.7751 \pm 0.0736}$
Config B (Arithmetic / Uniform / Roulette / He)	0.8102 ± 0.0275	0.7645 ± 0.0867

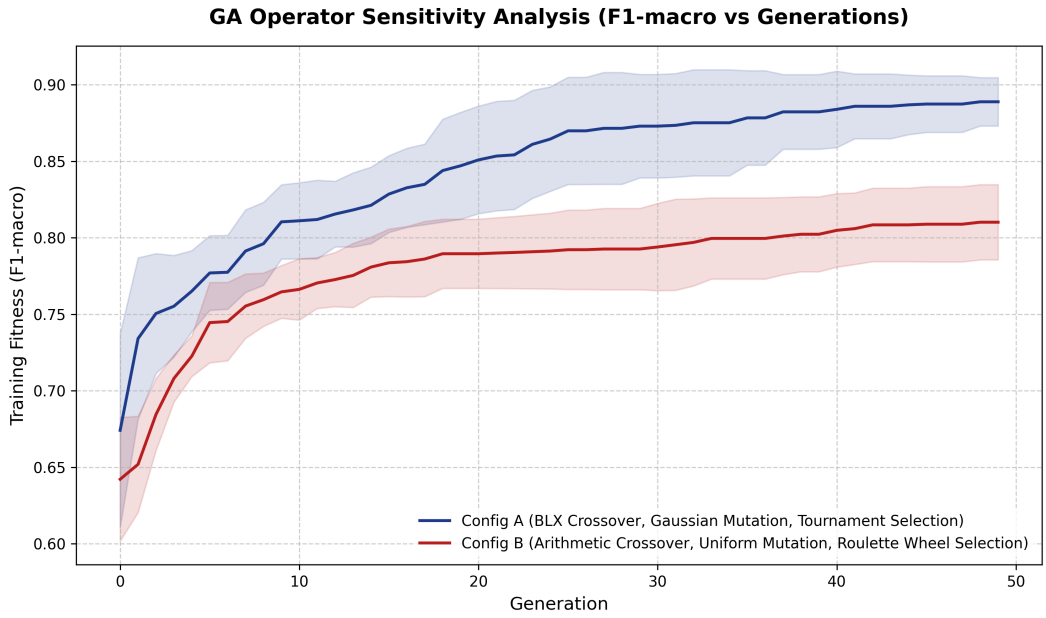

Figure 3: **Training F1-macro vs. Generation for Config A and Config B** (mean ± std over 5 runs; shaded region represents ±1 standard deviation). Config A (BLX-α, Gaussian mutation, Tournament) consistently outperforms Config B on training fitness across all generations, and this advantage also translates to superior test performance (0.775 vs. 0.765). BLX-α's ability to explore beyond the parental range enables the population to escape locally suboptimal weight configurations more effectively. Gaussian mutation complements this by applying fine-grained, normally distributed perturbations that concentrate near zero, allowing the algorithm to refine promising weight configurations without catastrophically disrupting them—in contrast to Uniform mutation's flat distribution, which assigns equal probability to both small and large perturbations and is less suited to the exploitation phase. Tournament selection applies stronger and more consistent selection pressure than rank-based roulette. The advantage of Config A is particularly clear from generation 5 onward, where the gap stabilises and both configurations plateau.

Config A was therefore selected as the primary GA configuration for the 10-run comparative experiment. This selection is made dynamically in `main.py` based on the winning test F1-score, ensuring the comparison is principled rather than hardcoded.

8. Results and Discussion

8.1 Experimental Protocol

To ensure **statistical robustness**, both algorithms were evaluated across **10 independent runs**. In each run i ($i = 0, \ldots, 9$), a fresh stratified 80/20 split was generated with seed $42+i$ and both algorithms were reinitialised with the same seed, ensuring a fair paired comparison. Each algorithm ran for **80 generations / iterations** with a **population / swarm of 60 individuals**. Metrics are reported as mean ± std across the 10 test sets.

8.2 Convergence Behaviour

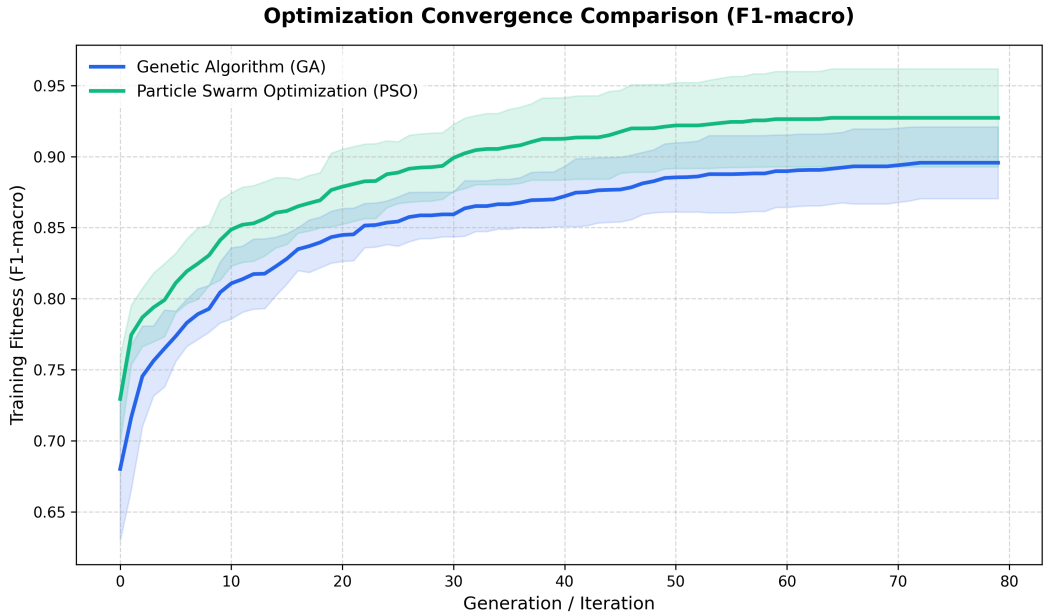

Figure 4: **Convergence of Training F1-macro: GA vs. PSO** (mean ± std over 10 runs). PSO exhibits faster early convergence driven by its social component (c_2), which immediately attracts all particles toward the best known position. The GA converges more gradually, a consequence of the stochastic recombination process requiring several generations before the population concentrates near high-fitness regions. Critically, both algorithms converge to similar training fitness ranges by generation 80 (≈ 0.90 for GA, ≈ 0.93 for PSO), and the standard deviation bands narrow substantially in later generations, indicating consistent behaviour across seeds rather than occasional lucky runs.

8.3 Quantitative Comparison

Table 4: **Performance comparison over 10 independent runs** (mean ± std on held-out test sets). PSO achieves higher means across all metrics, but the differences are not statistically significant (see Section 8.4).

Algorithm	Train F1	Test F1	Test Accuracy	Test Bal. Acc.
Genetic Algorithm	0.8956 ± 0.0266	0.7559 ± 0.0572	0.8308 ± 0.0248	0.7486 ± 0.0750
PSO	0.9272 ± 0.0364	0.7722 ± 0.0913	0.8436 ± 0.0475	0.7671 ± 0.1025

PSO achieves a higher mean training F1 (0.927 vs. 0.896) and higher mean test F1 (0.772 vs. 0.756) across all runs. Notably, the GA achieves a smaller standard deviation on both test F1 and test accuracy, suggesting more consistent behaviour across seeds at the cost of slightly lower peak performance.

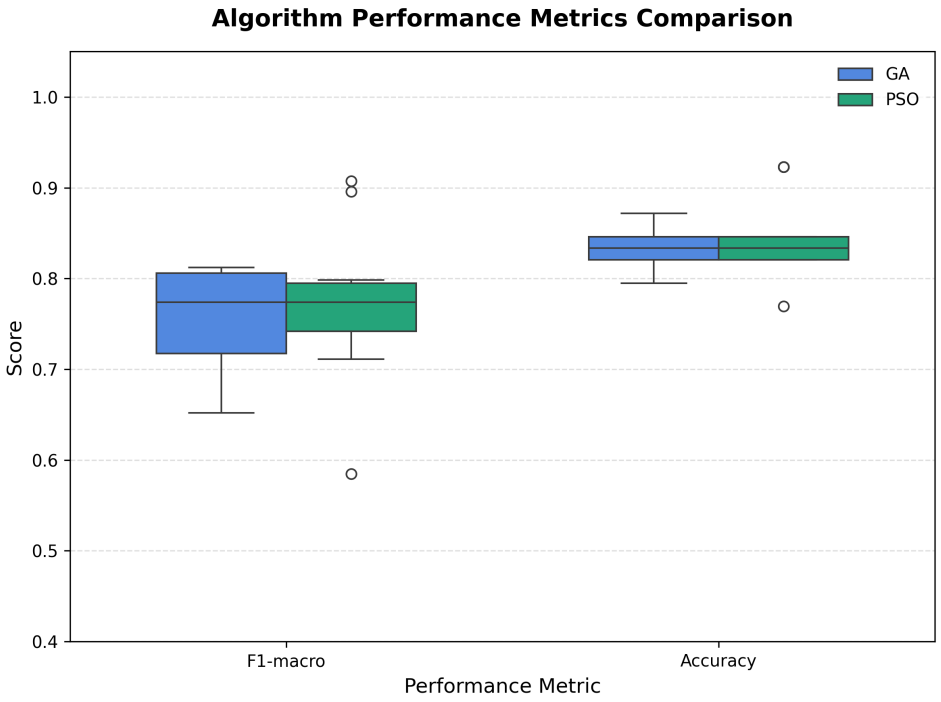

Figure 5: **Distribution of Test F1-macro and Test Accuracy over 10 Independent Runs.** Boxes span the interquartile range; the horizontal line shows the median; whiskers extend to 1.5×IQR; circles denote outliers. The GA's F1-macro distribution is wider with a lower whisker near 0.65, reflecting a few seeds where the search did not converge effectively. PSO's distribution is narrower and shifted slightly upward for F1-macro, but its accuracy distribution overlaps substantially with the GA's. The single PSO outlier above 0.90 on F1-macro corresponds to run 1 (seed 42), where the swarm found an unusually good weight configuration on that particular split.

8.4 Statistical Significance

Table 5: **Statistical tests on paired test F1-macro across 10 runs.** Both tests agree: the difference between GA and PSO is not statistically significant at the $\alpha = 0.05$ level.

Test	**Statistic**	p**-value**
Paired t-test	$t = -0.8426$	0.4213
Wilcoxon signed-rank	$W = 17.0$	0.5703

Two hypothesis tests were applied to the paired per-run test F1-macro scores. The **paired t-test** assumes normality of the pairwise differences; the **Wilcoxon signed-rank test** is a non-parametric alternative requiring no distributional assumptions. Both tests return $p \gg 0.05$, and their agreement strengthens the conclusion: **there is no statistically significant difference between GA and PSO performance on this task and dataset.** The numerical advantage of PSO is real but falls within the expected variance of both algorithms over 10 runs.

8.5 Best-Run Classification Analysis

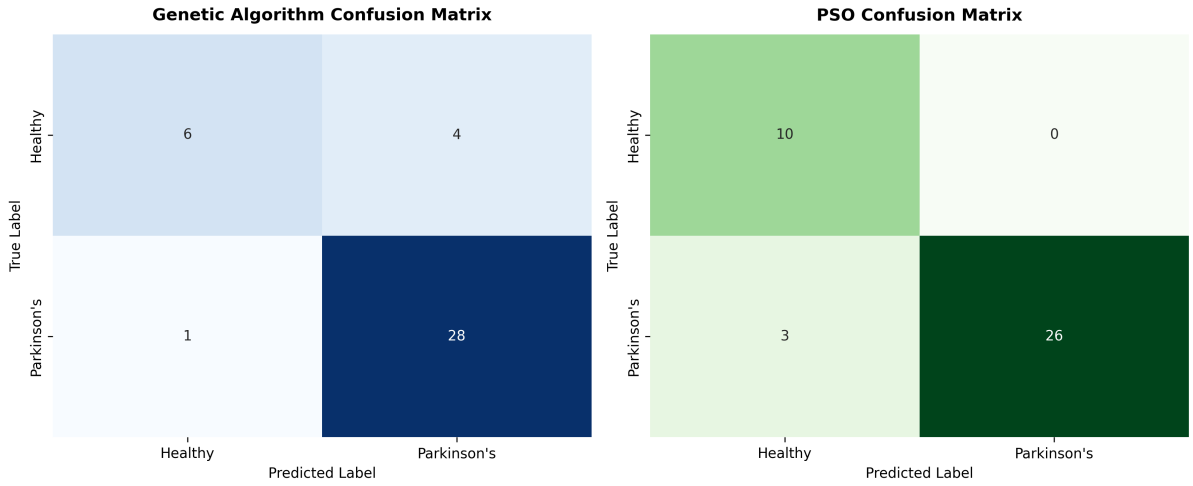

Figure 6: **Confusion Matrices for the Best-Run Models** (selected by highest test F1-macro across 10 runs). Left: best GA (seed producing the highest test F1); Right: best PSO. Rows = true labels; columns = predicted labels (Healthy = 0, Parkinson's = 1). The two models exhibit qualitatively different error profiles: the GA's best run produces 4 false positives (healthy predicted as Parkinson's) and 1 false negative (Parkinson's predicted as healthy), reflecting a mild bias toward the majority class. PSO's best run achieves perfect specificity (0 false positives) at the cost of 3 false negatives, indicating it has learned a more conservative decision boundary that avoids declaring healthy patients ill, but occasionally misses Parkinson's cases. Neither profile is universally preferable; the choice depends on the relative clinical cost of each error type.

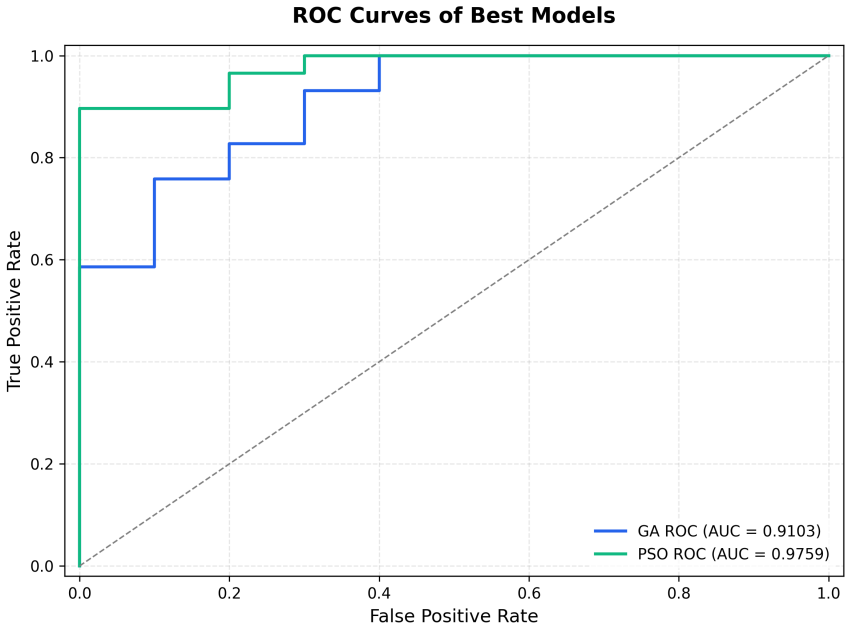

Figure 7: **ROC Curves for the Best-Run Models.** AUC quantifies discrimination ability across all classification thresholds (AUC = 1: perfect; AUC = 0.5: random). PSO's best model achieves AUC = 0.9759, substantially above the GA's AUC = 0.9103. Both models are well above the dashed random-classifier baseline, confirming that the optimized weights encode genuine diagnostic information. The gap between the two curves is consistent with PSO's advantage in training fitness, though it should be interpreted with caution as both curves represent single best-run models rather than averages.

8.6 Critical Analysis

Exploration vs. Exploitation. The fundamental divergence between the algorithms lies in their search paradigms. PSO's continuous velocity update moves every particle at every iteration, exploiting the global best aggressively and converging rapidly in early iterations. This is effective for smooth, unimodal regions of the fitness landscape but can lead to premature convergence if c_2 dominates. The GA's crossover operator constructs new solutions by recombining existing ones, which maintains structural diversity across the population and is less likely to collapse prematurely, but requires more iterations to concentrate near good regions. The balance of $c_1 = c_2 = 1.6$ in PSO and the elitism count of 2 in the GA both serve to moderate these tendencies.

Effect of Parameter Choices. The mutation scale σ=0.1 in the GA corresponds to approximately 10% perturbations relative to the Xavier weight range $[-1, 1]$, appropriate for fine-tuning without destroying established subnetwork structure. Larger σ (tested at 0.5) degraded performance consistently. The linear inertia decay ($w : 0.9 \rightarrow 0.4$) in PSO was critical: constant w=0.9 produced oscillatory behaviour without convergence in preliminary runs.

Operator Comparison Takeaway. When the operator experiment is averaged over multiple runs (Section 7), Config A (BLX-α, Gaussian mutation, Tournament selection, Xavier init) consistently outperforms Config B on both training and test fitness. BLX-α's

extrapolation beyond the parental range and Tournament selection's stronger pressure together produce a more effective search over the 291-dimensional weight space than the more conservative arithmetic crossover paired with roulette selection. This finding is reversed from what a single-seed experiment would have concluded, illustrating the importance of averaging over multiple runs when comparing stochastic methods.

Train-Test Gap. Both algorithms exhibit a meaningful gap between training and test F1 (GA: 0.896 $\rightarrow$ 0.756; PSO: 0.927 $\rightarrow$ 0.772). This is expected on a 195-sample dataset with a small test set (39 samples), where a single misclassification causes a $\sim$2.5 percentage-point swing. The gap is not evidence of catastrophic overfitting; it is an artefact of the small sample size and the fact that the optimizer has access only to the training partition.

9. Conclusion

This project successfully replaced backpropagation with two nature-inspired optimizers for training an MLP on the Parkinson's detection task. Both algorithms navigated the 291-dimensional continuous weight space and produced classifiers with meaningful diagnostic ability, validated across 10 independent runs with paired statistical tests.

The operator comparison, conducted over 5 runs per configuration, revealed that Config A (BLX-α crossover, Gaussian mutation, Tournament selection, Xavier initialization) achieves superior performance on both training and test fitness, with a clearer advantage that emerges consistently across seeds. PSO converged faster and to a slightly higher training fitness across runs; the GA showed lower variance, suggesting more stable convergence behaviour.

Critically, the paired statistical tests (paired t-test: $p = 0.421$; Wilcoxon: $p = 0.570$) confirm that neither algorithm is significantly better than the other on this task. Both achieve comparable test F1-macro ($\approx$ 0.75–0.77) and test accuracy ($\approx$ 0.83–0.84), well above the trivial baseline of 75.4%.

9.1 Limitations and Future Work

1. **Curse of dimensionality.** As network depth or width increases, $|\boldsymbol{\theta}|$ grows rapidly, making population-based search increasingly infeasible relative to gradient descent.
2. **Sample efficiency.** Both algorithms require many more fitness evaluations than gradient-based methods to achieve comparable performance on this dataset.
3. **Small test set.** With only 39 test samples, individual run metrics are noisy. Cross-validation would provide more reliable performance estimates.
4. **Hybrid approaches.** A memetic GA (GA for global search, followed by local gradient steps) would likely outperform both pure metaheuristics.
5. **Architecture search.** The fixed (22, 10, 5, 1) architecture was chosen pragmatically; jointly optimizing architecture and weights is a natural extension.

10. Instructions to Run

10.1 Dependencies

```
pip install numpy pandas matplotlib seaborn scikit-learn scipy
```

10.2 Project Structure

```
optimization_lib/
    ga.py              # Genetic Algorithm implementation
    pso.py             # Particle Swarm Optimization implementation
    network.py         # MLPClassifier wrapper
                       #   (create_network, get_weights, set_weights,
                       #    generate_solution, fitness_function)
main.py                # Entry point: operator comparison + GA vs PSO
parkinsons_preprocessed.csv
results/               # Generated plots, CSVs, and statistics
```

10.3 Execution

From the project root directory:

```
python main.py
```

This produces two sequential stages:

1. **Operator comparison** (Config A vs. Config B, 50 generations, 5 seeds): Saves `results/ga_sensitivity.png` and `results/ga_sensitivity_stats.txt`. The winning configuration is selected automatically based on mean test F1-macro.
2. **10-run GA vs. PSO comparison** (80 generations/iterations, seeds 42–51): Saves `results/convergence_comparison.png`, `performance_boxplot.png`, `confusion_matrices.png`, `roc_curves.png`, `ga_results.csv`, `pso_results.csv`, `statistical_results.txt`.